

Takeoff Kernels for Recursive Self-Improvement: A Short Formal Sketch

Jonathan R. Landers

July 2026

Abstract

Current theories of language-model self-improvement usually ask whether a generator can use its own verification signal to produce a better successor. That question records the amount or eventual limit of improvement, but not the finite shape by which improvement appears across rounds. This sketch applies the takeoff-kernel viewpoint to the standard generator–verifier–update loop. For bounded self-training, the kernel records what fraction of the eventual gain arrives in each round. For sustained recursive improvement, it records how log-capability growth approaches its terminal velocity. The resulting language distinguishes immediate, delayed, monotone, ringing, mixed, and diffusive takeoff even when two systems have the same terminal performance or asymptotic growth rate.

1 Context

Recent self-improvement theory studies a recurring pattern. A model generates candidate answers, a verifier scores or filters those answers, and the accepted data are distilled back into a new model. The verifier may be external, or it may be another use of the same model. Sharpening theory interprets the process as moving probability mass toward outputs the model can recognize as better [1]. The generation–verification gap measures whether such reweighting has positive value [2]. More recent analyses study the multi-round trajectory, finite-sample effects, saturation, and easy-to-hard curricula [3, 4].

This is usually bounded self-training, not unrestricted recursive self-improvement. Nevertheless, it already defines a discrete response curve over improvement rounds. The takeoff-kernel question is: not only *does* performance rise, but *when* and *in what shape* does the rise occur?

2 Standard self-improvement loop

Let $\mathcal{Q}$ be a question or task space, P a task distribution, and $\mathcal{A}$ an answer space. At round t :

$$q \sim P, \quad a \sim \pi_t(\cdot \mid q), \quad (1)$$

where π_t is the current solver. Let $s_t(q, a) \in [0, 1]$ be a verifier score and let w be a nonnegative filtering or weighting function. The verifier-induced target distribution is

$$\bar{\pi}_t(a \mid q) = \frac{\pi_t(a \mid q)w(s_t(q, a))}{Z_t(q)}, \quad (2)$$

where $Z_t(q)$ normalizes the distribution. Supervised fine-tuning or reinforcement learning then implements an approximate update

$$\pi_{t+1} = \mathcal{U}(\pi_t, \bar{\pi}_t; B_t), \quad (3)$$

with sample and compute budget B_t .

Let $r(q, a)$ denote the evaluation utility one ultimately cares about, which need not equal the verifier score, and define

$$J(\pi) = \mathbb{E}_{q \sim P, a \sim \pi(\cdot \mid q)}[r(q, a)], \quad J_t = J(\pi_t). \quad (4)$$

The idealized generation–verification advantage is

$$g_t = J(\bar{\pi}_t) - J(\pi_t). \quad (5)$$

A positive g_t says that verification exposes better probability mass. It does not say how efficiently finite training transfers that advantage into the successor. Schematically,

$$J_{t+1} - J_t \approx \eta_t g_t - \ell_t, \quad (6)$$

where η_t is uptake efficiency and ℓ_t collects sampling, optimization, distribution-shift, and verifier-mismatch losses.

3 Bounded improvement kernel

Suppose the repeated update converges to $J_\infty > J_0$. Define the normalized takeoff profile

$$F_t^b = \frac{J_t - J_0}{J_\infty - J_0}, \quad F_0^b = 0, \quad (7)$$

and its causal finite difference

$$\kappa_t^b = F_t^b - F_{t-1}^b, \quad t \geq 1. \quad (8)$$

Then

$$\sum_{t \geq 1} \kappa_t^b = 1. \quad (9)$$

Thus κ_t^b is the fraction of eventual capability gain delivered in round t . If performance improves monotonically, the kernel is nonnegative and can be read as a probability distribution over improvement time. If some entries are negative, the response is signed: the system overshoots, regresses, or rings before settling.

Two useful shape statistics transfer directly. For tolerance ε , the settling time is

$$T_\varepsilon = \min\{T : |F_t^b - 1| \leq \varepsilon \text{ for all } t \geq T\}, \quad (10)$$

and the excess signed response is

$$R(\kappa^b) = \sum_{t \geq 1} |\kappa_t^b| - 1. \quad (11)$$

The first measures how long improvement remains in startup; the second measures instability invisible in the final score.

4 Sustained recursive takeoff

Bounded benchmark accuracy cannot itself sustain a positive exponential velocity. For open-ended recursive improvement, choose an unbounded effective capability coordinate $C_t > 0$: for example, the size of the reliably solvable task frontier, inverse error $(1 - J_t)^{-1}$, or success odds where meaningful. Define the capability velocity

$$u_t = \log C_{t+1} - \log C_t. \quad (12)$$

If $u_t \rightarrow \alpha > 0$, define

$$F_t^r = \frac{u_t}{\alpha}, \quad \kappa_t^r = F_t^r - F_{t-1}^r. \quad (13)$$

This is the direct analogue of a velocity takeoff kernel for recursive redundancy [5]. The terminal number α says how fast capability eventually grows; the kernel says how that growth rate becomes visible. Therefore two systems can share the same α while having arbitrarily different finite takeoff. A genuinely accelerating regime in which u_t itself grows without bound would require a second-order extension rather than normalization by a fixed terminal velocity.

5 Takeoff types and interpretation

The same vocabulary now classifies self-improvement trajectories.

- *No takeoff.* Verification supplies no usable advantage, or learning cannot absorb it.
- *Immediate.* Most kernel mass lies in the first rounds; a strong verifier quickly extracts the available gain.
- *Delayed.* Early rounds contribute little until coverage, curriculum, or verification quality crosses a threshold.
- *Monotone long tail.* Gains stay positive but decay as the solver catches its verifier or the supply of accepted information is exhausted.
- *Ringling.* Negative kernel entries mark alternating improvement and regression from reward bias, distribution shift, or evaluator drift.
- *Mixed.* A fast output-level channel is followed by slower weight, tool, architecture, or meta-agent improvement.
- *Diffusive.* Algebraic rather than exponential settling suggests a broad or growing state space rather than finitely many dominant modes.

The generation–verification gap is therefore best viewed as an available forcing signal; the takeoff kernel is the realized temporal response. Under a local linear approximation, sequential stages such as verification, filtering, and distillation compose

by convolution, while parallel feedback channels mix. Outside that approximation the update is nonlinear, but the measured kernel still gives a model-independent summary of onset, delay, settling, and instability.

The basic separation principle is simple: terminal gain $J_\infty - J_0$, or terminal log-capability velocity α , does not determine takeoff. A fuller evaluation of recursive self-improvement should report the response shape as well: its kernel, settling time, signed response, and, where a modal fit is appropriate, its slowest settling mode.

References

- [1] A. Huang, A. Block, D. J. Foster, et al. Self-improvement in language models: The sharpening mechanism. *ICLR*, 2025. arXiv:2412.01951.
- [2] Y. Song, H. Zhang, C. Eisenach, S. Kakade, D. Foster, and U. Ghai. Mind the gap: Examining the self-improvement capabilities of large language models. *ICLR*, 2025. arXiv:2412.02674.
- [3] Y. Sun, Y. Liang, Z. Zhang, X. Liu, and J. Teng. Theoretical modeling of large language model self-improvement training dynamics through solver–verifier gap. *ICLR*, 2026. arXiv:2507.00075.
- [4] C. Liu, Y. Dong, Y. Shen, and Q. Lei. A task-centric theory for iterative self-improvement with easy-to-hard curricula. *ICLR*, 2026. arXiv:2602.10014.
- [5] J. R. Landers. The shape of recursive redundancy: Velocity takeoff kernels for memoization. Manuscript, 2026.